

Music Notation and Chord-Chart Formats for the OSMD and VexFlow Ecosystem

Executive summary and integration roadmap

A practical way to think about music content ingestion for apps is that you are really supporting **three different “worlds”**:

- **Full-score notation interchange** (multi-staff, polyphony, articulations, layout hints): dominated by **MusicXML** (plus its compressed container **MXL**) as a widely adopted interchange format maintained by the W3C ¹ Music Notation Community Group. ²
- **Rendered scholarly/edition encodings** (metadata-rich, editorial apparatus, critical editions): commonly represented with **MEI**, with fast browser rendering typically done via Verovio ³ . ⁴
- **Text-native performance charts** (lead sheets, chord charts, tabs): a mix of **ChordPro**, “Ultimate Guitar-style” chord sheets, OnSong-style text, and ASCII tablature conventions, with modern JS parsers such as ChordSheetJS ⁵ covering multiple variants. ⁶

Within the OSMD/VexFlow ecosystem specifically, it’s important to separate concerns:

- **VexFlow** is a **rendering engine** (TypeScript) that draws notation to Canvas/SVG; it does not define a standard interchange file format by itself. ⁷
- **OSMD** is a **MusicXML→renderer pipeline**: it parses MusicXML and uses VexFlow for engraving/layout, acting as a “missing link” between MusicXML interchange and VexFlow rendering. ⁸

Recommended first three formats to implement for broadest coverage

If your goal is maximum real-world coverage quickly (and you have no platform/budget constraints), the best “first 3” are:

1. **MusicXML (+ MXL)** for full notation and many lead-sheet style scores (including chord symbols and tablature constructs in MusicXML). ⁹
2. **ChordPro + Ultimate Guitar-style chord sheets** (text-based chord charts) because they dominate gig workflows and are easy to capture, edit, diff, version-control, and transpose. ¹⁰
3. **Guitar Pro family (GP3/GP4/GP5/GPX/GP)** because it’s a major tab ecosystem; when you support it you immediately unlock lots of guitar/bass/drum “pro tab” content and technique details. Rendering/parsing is well-covered across platforms by alphaTab ¹¹ . ¹²

Concise integration roadmap

A robust, format-agnostic integration roadmap looks like:

- **Detection**: sniff by extension + magic bytes (ZIP, XML prolog, etc.), and fall back to content heuristics (ChordPro directives, ABC headers like `X:` / `K:`, etc.). ¹³

- **Parsing:** parse into a **canonical internal model** (events/voices/measures for notation; sections/lines/chords for chord charts), keeping a lossless “source map” back to original tokens when possible (critical for editors). ¹⁴
- **Rendering:** choose the renderer per model:
 - MusicXML → OSMD → SVG/Canvas ¹⁵
 - MEI → Verovio → SVG ¹⁶
 - Guitar Pro → alphaTab → Canvas/SVG/audio playback ¹⁷
 - Chord charts → HTML/CSS (plus optional chord-diagram renderer) ¹⁸
- **Caching & performance:** cache parsed AST + rendered SVG per layout settings (zoom, page width), and stream large binaries (GP, MSCZ) rather than copying buffers blindly. OSMD explicitly notes that rendering “takes some time” and provides server-side/browserless rendering options. ¹⁹

OSMD and VexFlow ecosystem overview

VexFlow is a TypeScript library that renders music notation and guitar tablature to **HTML Canvas or SVG**, in browsers and Node.js contexts. ²⁰ Its native model is “constructive”: you build staves, voices, and notes via API calls (low-level) or via its higher-level helpers like **Factory** and **EasyScore**. ²¹

OSMD sits above that: it positions itself as the bridge between **MusicXML** and **VexFlow**, parsing MusicXML into a data model and then rendering via VexFlow (including features like tablature and many MusicXML tags, with some limitations). ²²

The VexFlow-adjacent “notation languages” you’ll see in the ecosystem fall into two categories:

- **VexFlow helper grammars:** *EasyScore* is a compact notation string grammar used to quickly create notes/voices without writing every object manually. ²¹
- **VexFlow-oriented text languages:** **VexTab** is a text-based language intended for “writeability” and supports guitar techniques such as hammer-ons/pull-offs/taps/slides in its syntax (e.g., `6h8p6`). It is described as an open specification, but its reference implementation has non-commercial licensing constraints. ²³

Entity table

Entity	What it is in this ecosystem	Primary role	License signal (for integration risk)
Verovio ³	MEI-first engraving toolkit with browser/WASM builds	Parse/render MEI (and some imports) → SVG	LGPLv3 (library), with separate tools having different licenses (e.g., editor prototype). ²⁴
alphaTab ¹¹	Cross-platform tab/notation renderer targeting Guitar Pro-family files	Parse Guitar Pro → render + playback ²⁵	MPL-2.0 (file-level copyleft). ²⁵

Entity	What it is in this ecosystem	Primary role	License signal (for integration risk)
abcjs ²⁶	Browser library for ABC notation rendering and MIDI generation	Parse ABC → render notation (+ options like tablature)	MIT. ²⁷
ChordSheetJS ⁵	Parser/formatter toolkit for chord sheets (ChordPro, UG-style, chords-over-words, etc.)	Parse chord charts → HTML/text/PDF	GPL-2.0. ²⁸
Library of Congress ²⁹	Preservation-oriented format notes for MusicXML/MEI	“Reality check” on format families, MIME types, sustainability	Not a code dependency; good for archival/metadata guidance. ³⁰

Database-ready catalog of relevant formats

This catalog emphasizes (1) OSMD/VexFlow adjacency, (2) text/binary chord/tab formats suitable for apps, and (3) primary sources for specs and MIME signals. MusicXML’s official media types are registered with IANA, and the W3C MusicXML documentation specifies recommended extensions and the compressed MXL container structure. ³¹ ChordPro’s spec and common extensions are documented by the ChordPro project itself, and OnSong documents both bracketed chords and “chords over lyrics” ingestion patterns used widely in the field. ³² ABC’s standard documents chord symbols and lyric alignment (`w:`), and IANA registers `text/vnd.abc` as an ABC media type. ³³

Format records

Each record below is intended to map cleanly into a database row (columns: **extensions**, **mime_types**, boolean capability flags, **recommended_js_ts**, **license**, **parser_complexity**, etc.).

Format	Text / Binary	Common extensions	Common MIME types (official where known)	Primary/official spec or reference	O re
MusicXML (uncompressed)	Text (XML)	<code>.musicxml</code> (recommended), legacy <code>.xml</code> ³⁴	<code>application/vnd.recordare.musicxml+xml</code> (IANA) ³⁵	W3C MusicXML reference & tutorials ³⁶	N O pa M re Ve
MXL (compressed MusicXML)	ZIP container (binary) containing XML	<code>.mxl</code> ⁴³	<code>application/vnd.recordare.musicxml</code> (IANA) ⁴⁴	W3C “Compressed .MXL Files” tutorial + container structure ⁴⁵	Sa M (C M ty un pa
MEI	Text (XML)	<code>.mei</code> (widely used) ⁵⁰	No IANA registration; commonly treated as XML; some ecosystems use unregistered <code>application/vnd.mei+xml</code> ⁵¹	MEI project + guidelines ⁵²	U O na re Ve co pi (M to se

Format	Text / Binary	Common extensions	Common MIME types (official where known)	Primary/official spec or reference	O re
MuseScore internal formats	Mixed: XML + ZIP container	<code>.mscx</code> (XML), <code>.mscz</code> (ZIP of <code>.mscx</code>) ⁵⁹	Typically <code>application/zip</code> or <code>application/xml</code> (no standard MIME) ⁶⁰	MuseScore handbook notes MSCX is not stable across versions and recommends MusicXML for interchange ⁵⁹	In M M O
ABC notation	Text	<code>.abc</code> ⁶⁴	<code>text/vnd.abc</code> (IANA) ⁶⁵	ABC standard v2.1 (notably: chord symbols and w: lyrics alignment) ⁶⁶	N Ve ty re ab co el
LilyPond	Text	<code>.ly</code> (convention) ⁷⁵	Typically <code>text/plain</code>	LilyPond manuals for input structure and compilation ⁷⁶	N Ve co ap co Li SV fr us m 7

Format	Text / Binary	Common extensions	Common MIME types (official where known)	Primary/official spec or reference	Other
ChordPro (v6 ecosystem)	Text	Common: <code>.cho</code> , <code>.crd</code> , <code>.chopro</code> , <code>.chord</code> , <code>.pro</code> typically <code>text/plain</code> 82	<code>text/plain</code>	ChordPro introduction + directives spec 83	In O spec di ch tr le la
Ultimate Guitar-style chord sheets	Text	Usually <code>.txt</code> or site/app content	<code>text/plain</code>	De facto format; supported explicitly by ChordSheetJS "UltimateGuitarParser" 28	U O
ASCII "chords over lyrics"	Text	<code>.txt</code> , <code>.tab</code> sometimes 88	<code>text/plain</code>	Convention (not a formal spec); OnSong documents ingestion and conversion	In

Format	Text / Binary	Common extensions	Common MIME types (official where known)	Primary/official spec or reference	O re
Inline-bracket chords (ChordPro-style without directives)	Text	<code>.txt</code>	<code>text/plain</code>	Convention; OnSong notes bracketed chords are its internal standard and also used by ChordPro ⁹⁰	In
OnSong text format	Text	<code>.onsong</code> ⁹³	<code>text/plain</code>	OnSong format docs (metadata section + chord handling) ⁹⁴	In
OnSong Archive / Backup	Binary container	<code>.archive</code> ⁹⁹ , <code>.onsongarchive</code> App-specific, <code>.backup</code>		OnSong docs explicitly state archive is binary and only readable by OnSong ¹⁰⁰	U
OpenSong song XML	XML	Often no extension; OpenSongApp mentions <code>.ost</code> or none; OpenSong desktop docs describe XML layout ¹⁰²	<code>application/xml</code> (typical) ⁶⁰	OpenSong file formats page (XML structure) ¹⁰³	In

Format	Text / Binary	Common extensions	Common MIME types (official where known)	Primary/official spec or reference	O re
OpenLyrics	XML	<code>.xml</code> commonly	<code>application/xml</code> (typical) ⁶⁰	OpenLyrics docs (features include chords + schema) ¹⁰⁷	In
SongBook (LinkeSOFT)	Text (ChordPro usage) + tabs	<code>.pro</code> , <code>.chopro</code> , <code>.txt</code> , <code>.tab</code> documented by vendor ¹¹²	<code>text/plain</code>	Vendor documentation states SongBook stores songs in ChordPro format and supports tab files ¹¹³	In
SongbookPro database export	Mixed (app formats + imports)	<code>.sbp</code> , <code>.sbpbackup</code> + imports ChordPro/OnSong exports ¹¹⁵	App-specific	SongbookPro docs list supported import formats and proprietary backups ¹¹⁵	In

Format	Text / Binary	Common extensions	Common MIME types (official where known)	Primary/official spec or reference	O re
Guitar Pro 3–5	Binary	<code>.gp3</code> ¹¹⁸ , <code>.gp4</code> , <code>.gp5</code>	App-specific	No public official spec; supported by alphaTab	In
Guitar Pro 6–8 family	Mixed (often ZIP/ XML + binary variants)	<code>.gpx</code> , <code>.gp</code> (and related) ¹²⁴	App-specific	Vendor-defined; widely used; alphaTab supports many formats	In
Power Tab (.ptb)	Binary	<code>.ptb</code> ¹²⁸	App-specific	Power Tab developer/ forum references exist; Power Tab dev center documents features like bends; third-party libs exist ¹²⁹	In

Format	Text / Binary	Common extensions	Common MIME types (official where known)	Primary/official spec or reference	O re
Humdrum / **kern	Text	.krn recommended ¹³³	text/plain	Humdrum **kern reference (file extension guidance) ¹³⁴	In co re vi H ec
VexTab	Text	Often embedded; no universal extension; commonly stored as .txt or embedded blocks ¹³⁸	text/plain	VexTab tutorial (syntax incl. h/p/t/s techniques) ¹³⁹	V a V te re
iReal Pro "irealb://" chord charts	Text-like encoded string in URL	irealb:// links (export format) ¹⁴⁴	Custom URL scheme	iReal Pro developer docs for embedding/export links ¹⁴⁴	U

Format	Text / Binary	Common extensions	Common MIME types (official where known)	Primary/official spec or reference	O re
MNX (emerging)	Likely text (JSON/XML-like spec evolution)	TBD	TBD	W3C MNX community spec drafts ¹⁴⁸	Fu no na
GUIDO Music Notation (GMN)	Text	Typically <code>.gmh</code> or <code>.gmj</code> or <code>.gm</code>	<code>text/plain</code>	Guido format docs describe GMN as a plain-text formal language for score-level representation ¹⁴⁹	U
LRC v2 draft (syncd lyrics w/ chords)	Text	Typically <code>.lrc</code> variants	<code>text/plain</code>	Draft spec in public repo mentions chords + metadata ambitions ¹⁵²	U

Comparative matrix for format selection

This matrix is optimized for “what should I implement next?” decisions, especially for web/TypeScript stacks.

Format	Text / Binary	Chord support	Lyrics support	Tablature support	Technique specificity (bends/slides/etc.)	Parse complexity	Strong JS/TS parser available?	Recommend JS/TS support
MusicXML	Text (XML)	Yes (<code><harmony></code>) 39	Yes (tooling extracts lyrics; OSMD can hide lyrics) 41	Yes (tablature examples in spec) 40	Yes (e.g., hammer-on/pull-off elements) 154	High	Yes	OSMD
MXL	ZIP+XML	Yes	Yes	Yes	Yes	Medium	Yes	OSMD contains handling
MEI	Text (XML)	Yes (encoding-dependent)	Yes	Yes (stringtab modules) 157	Yes (encoding-dependent)	High	Yes	Verovis
ABC	Text	Yes (chord symbols in standard) 69	Yes (<code>w: / W:</code>) 70	Limited; abcs offers tablature rendering option 71	Limited (extensions vary)	Medium	Yes	abcs
ChordPro	Text	Yes	Yes	ASCII tab environments in ecosystem tooling 85	Low semantic technique (tab is free-form)	Low–Medium	Yes	ChordPro 28
UG-style chord sheets	Text	Yes	Yes	Sometimes ASCII tab blocks	Low semantic technique	Low	Yes	ChordPro UG parser
OnSong text	Text	Yes	Yes	Via tab-like sections	Low semantic technique	Low–Medium	Indirect	Parse ChordPro metadata

Format	Text / Binary	Chord support	Lyrics support	Tablature support	Technique specificity (bends/slides/etc.)	Parse complexity	Strong JS/TS parser available?	Recommendation JS/TS support
Guitar Pro family	Binary / mixed	Yes	Yes (varies)	Yes	Strong technique support (alphaTab docs list effects) 121	Medium-High	Yes	alphaTab
Power Tab (.ptb)	Binary	Limited	Limited	Yes	Medium	Medium	Indirect	alphaTab import
**kern	Text	Varies	Varies	Not primary	Varies	Medium	Indirect	Verovini Humdrum toolchain plugin ecosystem 160
VexTab	Text	Yes (depends on usage)	Limited	Yes	Explicit (h/p/t/s) 141	Low-Medium	Yes	vexflow VexFlow
iReal Pro links	Encoded text	Yes	No	No	No	Medium	Not mainstream in JS	Use no parser reimplement grammar guided docs

Parser and renderer integration patterns with code examples

Reference integration flow (mermaid)

```

flowchart LR
    A[User uploads / selects file] --> B[Detect format]
    B -->|.musicxml .xml| C1[Parse MusicXML]
    B -->|.mxl| C2[Unzip + container.xml -> MusicXML]
    B -->|.mei| C3[Parse MEI]
    B -->|.gp3/.gp4/.gp5/.gpx/.gp| C4[Parse Guitar Pro]
    B -->|ChordPro / UG / OnSong text| C5[Parse chord chart text]
    B -->|.abc| C6[Parse ABC]

```

```
C1 --> R1[OSMD render]
C2 --> R1
C3 --> R2[Verovio render]
C4 --> R3[alphaTab render/playback]
C5 --> R4[HTML chord chart renderer]
C6 --> R5[abcjs render]
R1 --> Z[SVG/Canvas + interaction layer]
R2 --> Z
R3 --> Z
R4 --> Z
R5 --> Z
```

Canonical model strategy (mermaid)

A pragmatic “database-ready” internal model often benefits from splitting into two normalized domains:

```
classDiagram
class ScoreDocument {
  +id: string
  +sourceFormat: string
  +metadata: Map
  +parts: Part[]
}
class Part {
  +id: string
  +name: string
  +staves: Staff[]
}
class Staff {
  +measures: Measure[]
}
class Measure {
  +events: Event[]
}
class Event {
  +type: string
  +time: rational
  +payload: any
}

class ChartDocument {
  +id: string
  +sourceFormat: string
  +metadata: Map
  +sections: Section[]
}
```

```

class Section {
  +label: string
  +lines: ChartLine[]
}
class ChartLine {
  +tokens: Token[]
}
class Token {
  +kind: chord|lyric|comment|tab
  +text: string
}

ScoreDocument --> Part
Part --> Staff
Staff --> Measure
Measure --> Event

ChartDocument --> Section
Section --> ChartLine
ChartLine --> Token

```

Code examples

The snippets below are intentionally “integration-shaped”: they show parsing + rendering entry points you can wrap behind adapters.

MusicXML / MXL → OSMD render (TypeScript)

OSMD is designed as the “missing link” between MusicXML and VexFlow and is available as an npm module; it renders MusicXML to SVG/Canvas and can output SVG/PNG even server-side. ¹⁶³

```

import { OpenSheetMusicDisplay } from "opensheetmusicdisplay";

async function renderMusicXML(urlOrFile: string | File, container: HTMLElement)
{
  const osmd = new OpenSheetMusicDisplay(container, {
    backend: "svg",
    drawTitle: true,
  });

  if (typeof urlOrFile === "string") {
    await osmd.load(urlOrFile); // can be .musicxml, .xml, or .mxl depending on
server setup
  } else {
    const data = await urlOrFile.arrayBuffer();
    await osmd.load(data); // ArrayBuffer path is handy for uploads and .mxl

```

```

    }

    await osmd.render();
    return osmd;
}

```

MEI → Verovio → SVG (TypeScript)

Verovio is an LGPL-licensed engraving toolkit that renders MEI to SVG and provides JavaScript/WASM distributions. ¹⁶⁴

```

import verovio from "verovio";

async function renderMEIToSVG(meIxmlText: string, target: HTMLElement) {
    const toolkit = new verovio.toolkit();
    toolkit.setOptions({
        scale: 40,
        footer: "none",
        header: "none",
    });

    toolkit.loadData(meIxmlText);
    const svg = toolkit.renderToSVG(1); // page 1
    target.innerHTML = svg;
}

```

ABC → abcjs (TypeScript)

ABC is a text-based notation standard with a registered MIME type `text/vnd.abc`, and abcjs renders it directly in the browser under MIT license. ¹⁶⁵

```

import { renderAbc } from "abcjs";

function renderABC(abcText: string, targetElementId: string) {
    renderAbc(targetElementId, abcText, {
        responsive: "resize",
        add_classes: true,
    });
}

```


ChordPro / Ultimate Guitar / chords-over-lyrics → ChordSheetJS (TypeScript)

ChordSheetJS supports multiple “real-world” chord sheet dialects (ChordPro, Ultimate Guitar style, and chords-over-words) and can format as HTML/text; it also highlights that measured layout and PDF export are beta. ¹⁶⁶

```
import ChordSheetJS from "chordsheetjs";

type InputKind = "chordpro" | "ultimateGuitar" | "chordsOverWords";

function parseChordChart(text: string, kind: InputKind) {
  const parser =
    kind === "chordpro"
      ? new ChordSheetJS.ChordProParser()
      : kind === "ultimateGuitar"
        ? new ChordSheetJS.UltimateGuitarParser()
        : new ChordSheetJS.ChordsOverWordsParser();

  return parser.parse(text);
}

function formatChordChartAsHTML(song: any) {
  const formatter = new ChordSheetJS.HtmlDivFormatter();
  return formatter.format(song);
}
```

Guitar Pro (GP3/4/5/GPX/GP) → alphaTab (TypeScript)

alphaTab provides web installation paths and documents rich technique/effect rendering, including bends and slides. ¹⁶⁷

```
import { alphaTab } from "@coderline/alphatab";

async function renderGuitarPro(file: File, host: HTMLElement) {
  const settings = new alphaTab.Settings();
  settings.player.enablePlayer = true;

  const api = new alphaTab.AlphaTabApi(host, settings);

  const data = await file.arrayBuffer();
  api.load(data);

  return api; // provides events for scrolling/playback integration
}
```

VexTab → VexFlow rendering (JavaScript/TypeScript-shaped)

VexTab’s tutorial documents technique tokens (hammer-ons/pull-offs/taps/slides) and the VexFlow site notes non-commercial constraints on the reference implementation. ¹⁶⁸

```
// Pseudocode: vextab reference API varies by distribution.
// Conceptually: parse VexTab -> generate VexFlow objects -> draw.

function renderVexTab(vextabText: string, canvasOrDiv: HTMLElement) {
  // const tab = new VexTab();
  // tab.parse(vextabText);
  // tab.render(canvasOrDiv);
}
```

Emerging and ad-hoc formats worth tracking

This section lists formats that commonly appear in the wild even when they’re not “official standards,” plus a few emerging targets.

Ad-hoc tablature conventions you will routinely ingest

If you accept pasted tabs from forums, messaging apps, or PDFs-transcribed-to-text, you’ll see recurring ASCII patterns:

- **Six-line string staff** (EADGBE) with fret numbers; technique letters like **H** / **P** / **B** are common conventions in instructional materials. ¹⁶⁹
- **Symbols for bends/slides/hammer-ons** vary across communities and editors; many readers treat them as interpretive rather than strictly machine-parseable. ¹⁷⁰

Pragmatic ingestion strategy: treat ASCII tab as **semi-structured text**, tokenize lines, and only promote to semantic events (bend/slide/hammer) when the pattern is unambiguous—otherwise preserve as “verbatim tab block” in your internal model.

Conversions as a first-class capability (often more valuable than parsing everything)

A recurring theme across these ecosystems is that **conversion is often more stable than native parsing**:

- W3C MusicXML docs describe the compressed MXL container and recommended extensions, making MusicXML an attractive “hub” format for interchange. ⁴⁹
- LilyPond explicitly ships `musicxml2ly` to import MusicXML into LilyPond, reinforcing that MusicXML is treated as a practical interchange entry point. ¹⁷¹
- Verovio offers MusicXML→MEI conversion (with limitations) and broader conversion pipelines are discussed in its documentation. ¹⁷²

Formats to “watch” rather than “support first”

- **MNX** is a W3C-community effort and a plausible future interchange target, but it remains a moving target compared with MusicXML's established ecosystem. ¹⁴⁸
- **LRC v2 drafts** demonstrate growing interest in timed lyrics + chord overlays, but licensing constraints and spec volatility make it a specialized choice. ¹⁵²

Licensing reality check for app integration

Licensing is often the hidden constraint in “format support”:

- OSMD is BSD-3-Clause. ¹⁵⁵
- VexFlow is MIT. ¹⁷³
- abcjs is MIT. ⁷²
- alphaTab is MPL-2.0 (file-level copyleft). ¹⁷⁴
- ChordSheetJS is GPL-2.0 (strong copyleft), which can be a non-starter for some proprietary distribution models unless isolated appropriately. ²⁸
- Verovio is LGPLv3, with guidance emphasizing dynamic linking and non-modification scenarios for proprietary products. ¹⁷⁵

In practice, this pushes many teams toward an adapter architecture: you wrap each parser/renderer behind an internal interface, and you can swap implementations if licensing or performance changes later—without rewriting your product's domain model.

¹ ¹⁷² <https://www.verovio.org/musicxml.html>

<https://www.verovio.org/musicxml.html>

² ⁹ ¹³ ⁴⁵ ⁴⁷ ⁴⁸ ⁴⁹ ¹⁵⁶ <https://www.w3.org/2021/06/musicxml40/tutorial/compressed-mxl-files/>

<https://www.w3.org/2021/06/musicxml40/tutorial/compressed-mxl-files/>

³ ⁸⁶ <https://raw.githubusercontent.com/ChordPro/chordpro/master/LICENSE>

<https://raw.githubusercontent.com/ChordPro/chordpro/master/LICENSE>

⁴ ¹⁶ <https://www.verovio.org/>

<https://www.verovio.org/>

⁵ ⁶⁵ ⁷³ ¹⁶⁵ <https://www.iana.org/assignments/media-types/text/vnd.abc>

<https://www.iana.org/assignments/media-types/text/vnd.abc>

⁶ ¹⁴ ¹⁸ ²⁸ ⁸⁵ ⁸⁷ ¹²⁹ ¹⁶⁶ <https://github.com/martijnversluis/ChordSheetJS>

<https://github.com/martijnversluis/ChordSheetJS>

⁷ ²⁰ ²¹ <https://github.com/vexflow/vexflow>

<https://github.com/vexflow/vexflow>

⁸ ¹⁵ ¹⁹ ²² ³⁸ ¹⁶³ <https://github.com/opensheetmusicdisplay/opensheetmusicdisplay>

<https://github.com/opensheetmusicdisplay/opensheetmusicdisplay>

¹⁰ ³² ⁸² ⁸³ <https://www.chordpro.org/chordpro/chordpro-introduction/>

<https://www.chordpro.org/chordpro/chordpro-introduction/>

11 24 53 54 57 58 164 <https://github.com/rism-digital/verovio>
<https://github.com/rism-digital/verovio>

12 <https://www.guitar-pro.com/>
<https://www.guitar-pro.com/>

17 119 125 132 159 167 <https://alphatab.net/docs/getting-started/installation-web>
<https://alphatab.net/docs/getting-started/installation-web>

23 139 141 168 <https://www.vexflow.com/vextab/tutorial.html>
<https://www.vexflow.com/vextab/tutorial.html>

25 122 123 126 127 174 <https://raw.githubusercontent.com/CoderLine/alphaTab/develop/LICENSE>
<https://raw.githubusercontent.com/CoderLine/alphaTab/develop/LICENSE>

26 169 170 <https://www.guitartricks.com/blog/guitar-tab-symbols>
<https://www.guitartricks.com/blog/guitar-tab-symbols>

27 72 <https://raw.githubusercontent.com/paulrosen/abcjs/main/LICENSE.md>
<https://raw.githubusercontent.com/paulrosen/abcjs/main/LICENSE.md>

29 75 <https://usermanuals.musicxml.com/MusicXML/Content/EL-MusicXML.htm>
<https://usermanuals.musicxml.com/MusicXML/Content/EL-MusicXML.htm>

30 <https://www.loc.gov/preservation/digital/formats/fdd/fdd000503.shtml>
<https://www.loc.gov/preservation/digital/formats/fdd/fdd000503.shtml>

31 35 <https://www.iana.org/assignments/media-types/application/vnd.recordare.musicxml%2Bxml>
<https://www.iana.org/assignments/media-types/application/vnd.recordare.musicxml%2Bxml>

33 66 69 70 74 <https://abcnotation.com/wiki/abc%3Astandard%3Av2.1>
<https://abcnotation.com/wiki/abc%3Astandard%3Av2.1>

34 36 43 <https://www.w3.org/2021/06/musicxml40/container-reference/elements/container/>
<https://www.w3.org/2021/06/musicxml40/container-reference/elements/container/>

37 41 171 <https://lilypond.org/doc/v2.25/Documentation/usage/invoking-musicxml2ly>
<https://lilypond.org/doc/v2.25/Documentation/usage/invoking-musicxml2ly>

39 <https://www.w3.org/2021/06/musicxml40/musicxml-reference/elements/harmony/>
<https://www.w3.org/2021/06/musicxml40/musicxml-reference/elements/harmony/>

40 62 <https://w3c.github.io/musicxml/musicxml-reference/examples/tutorial-tablature/>
<https://w3c.github.io/musicxml/musicxml-reference/examples/tutorial-tablature/>

42 <https://en.wikipedia.org/wiki/MusicXML>
<https://en.wikipedia.org/wiki/MusicXML>

44 <https://www.iana.org/assignments/media-types/>
<https://www.iana.org/assignments/media-types/>

46 <https://app.unpkg.com/opensheetmusicdisplay%401.1.0/files/package.json>
<https://app.unpkg.com/opensheetmusicdisplay%401.1.0/files/package.json>

50 51 <https://omeka.org/s/modules/Verovio/>
<https://omeka.org/s/modules/Verovio/>

52 <https://music-encoding.org/>

<https://music-encoding.org/>

55 <https://www.loc.gov/preservation/digital/formats/fdd/fdd000502.shtml>

<https://www.loc.gov/preservation/digital/formats/fdd/fdd000502.shtml>

56 157 https://music-encoding.org/archive/comparison-5_1.html

https://music-encoding.org/archive/comparison-5_1.html

59 61 63 <https://www.power-tab.net/developers.php>

<https://www.power-tab.net/developers.php>

60 https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/MIME_types/Common_types

https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/MIME_types/Common_types

64 <https://abcmusicnotation.weebly.com/abc-files.html>

<https://abcmusicnotation.weebly.com/abc-files.html>

67 71 130 131 <https://github.com/paulrosen/abcjs>

<https://github.com/paulrosen/abcjs>

68 <https://abcnotation.com/>

<https://abcnotation.com/>

76 79 <https://lilypond.org/doc/v2.24/Documentation/learning/compiling-a-file>

<https://lilypond.org/doc/v2.24/Documentation/learning/compiling-a-file>

77 81 <https://lilypond.org/doc/v2.25/Documentation/usage-big-page.html>

<https://lilypond.org/doc/v2.25/Documentation/usage-big-page.html>

78 <https://www.npmjs.com/package/lilynode>

<https://www.npmjs.com/package/lilynode>

80 <https://lilypond.org/download.html>

<https://lilypond.org/download.html>

84 <https://www.chordpro.org/chordpro/chordpro-directives/>

<https://www.chordpro.org/chordpro/chordpro-directives/>

88 112 <https://linksoft.com/songbook/win10touch.html>

<https://linksoft.com/songbook/win10touch.html>

89 <https://onsongapp.com/docs/features/formats/onsong/chords/>

<https://onsongapp.com/docs/features/formats/onsong/chords/>

90 91 92 <https://www.onsongapp.com/docs/chords-bracketed>

<https://www.onsongapp.com/docs/chords-bracketed>

93 100 101 <https://onsongapp.com/docs/features/output/file-formats>

<https://onsongapp.com/docs/features/output/file-formats>

94 95 96 97 <https://www.onsongapp.com/docs/features/formats/onsong/>

<https://www.onsongapp.com/docs/features/formats/onsong/>

98 <https://www.onsongapp.com/docs/features/formats/onsong/section>

<https://www.onsongapp.com/docs/features/formats/onsong/section>

99 <https://onsongapp.com/docs/features/formats/onsong-archive>
<https://onsongapp.com/docs/features/formats/onsong-archive>

102 105 <https://www.opensongapp.com/user-manual/songs/create-new-song>
<https://www.opensongapp.com/user-manual/songs/create-new-song>

103 <https://opensong.org/development/file-formats/>
<https://opensong.org/development/file-formats/>

104 106 <https://github.com/Gajusbonus/SongConverter>
<https://github.com/Gajusbonus/SongConverter>

107 109 110 <https://docs.openlyrics.org/>
<https://docs.openlyrics.org/>

108 <https://docs.openlyrics.org/en/latest/conversion.html>
<https://docs.openlyrics.org/en/latest/conversion.html>

111 <https://github.com/openlyrics/openlyrics/issues/52>
<https://github.com/openlyrics/openlyrics/issues/52>

113 114 <https://linksoft.com/songbook/chordproformat.html>
<https://linksoft.com/songbook/chordproformat.html>

115 116 117 <https://songbook-pro.com/docs/manual/adding-songs/>
<https://songbook-pro.com/docs/manual/adding-songs/>

118 120 https://pyguitarpro.readthedocs.io/_/downloads/en/v0.4/pdf/
https://pyguitarpro.readthedocs.io/_/downloads/en/v0.4/pdf/

121 <https://alphatab.net/docs/reference/score>
<https://alphatab.net/docs/reference/score>

124 128 <https://goplayalong.com/help/download/>
<https://goplayalong.com/help/download/>

133 <https://www.humdrum.org/rep/kern/>
<https://www.humdrum.org/rep/kern/>

134 <https://www.humdrum.org/Humdrum/representations/kern.html>
<https://www.humdrum.org/Humdrum/representations/kern.html>

135 <https://github.com/humdrum-tools/humdrum-notation-plugin/blob/master/example.krn>
<https://github.com/humdrum-tools/humdrum-notation-plugin/blob/master/example.krn>

136 <https://wiki.ccarh.org/images/6/6e/Humdrum-File-Format.pdf>
<https://wiki.ccarh.org/images/6/6e/Humdrum-File-Format.pdf>

137 160 <https://verovio.humdrum.org/>
<https://verovio.humdrum.org/>

138 <https://justsolve.archiveteam.org/wiki/VexTab>
<https://justsolve.archiveteam.org/wiki/VexTab>

140 161 <https://github.com/0xfe/vexTab>
<https://github.com/0xfe/vexTab>

142 143 <https://vexflow.com/vextab/>

<https://vexflow.com/vextab/>

144 147 162 <https://www.irealpro.com/developer-docs>

<https://www.irealpro.com/developer-docs>

145 <https://docs.rs/ireal-parser>

<https://docs.rs/ireal-parser>

146 <https://songbook-pro.com/docs/getting-started/importing-songs/>

<https://songbook-pro.com/docs/getting-started/importing-songs/>

148 <https://www.w3.org/community/music-notation/>

<https://www.w3.org/community/music-notation/>

149 151 <https://guidolib.sourceforge.io/doc/guidolib/>

<https://guidolib.sourceforge.io/doc/guidolib/>

150 <https://github.com/grame-cncm/guidolib>

<https://github.com/grame-cncm/guidolib>

152 153 <https://github.com/marz1877/LRCv2>

<https://github.com/marz1877/LRCv2>

154 <https://www.w3.org/2021/06/musicxml40/musicxml-reference/elements/hammer-on/>

<https://www.w3.org/2021/06/musicxml40/musicxml-reference/elements/hammer-on/>

155 <https://raw.githubusercontent.com/opensheetmusicdisplay/opensheetmusicdisplay/develop/LICENSE>

<https://raw.githubusercontent.com/opensheetmusicdisplay/opensheetmusicdisplay/develop/LICENSE>

158 175 <https://book.verovio.org/introduction/licensing.html>

<https://book.verovio.org/introduction/licensing.html>

173 <https://raw.githubusercontent.com/vexflow/vexflow/master/LICENSE>

<https://raw.githubusercontent.com/vexflow/vexflow/master/LICENSE>